

Two vertical lines, one blue and one orange, are positioned on the left side of the slide.

GPU Architecture



MORGAN & CLAYPOOL PUBLISHERS

Hello. [Sign in](#) to personalize your visit. New user? [Register now](#).
[Home](#) | [Synthesis](#) | [Colloquium](#) | [Search](#) | [Profile](#) | [Author](#) | [Help](#) | [About](#) | [Bookstore](#)
Quick search: within: [go](#)

General-Purpose Graphics Processor Architectures

[Synthesis Lectures on Computer Architecture](#)

May 2018, 140 pages, (<https://doi.org/10.2200/S00848ED1V01Y201804CAC044>)

Tor M. Aamodt
University of British Columbia

Wilson Wai Lun Fung
Samsung Electronics

Timothy G. Rogers
Purdue University

Abstract

Originally developed to support video games, graphics processor units (GPUs) are now increasingly used for general-purpose (non-graphics) applications ranging from machine learning to mining of cryptographic currencies. GPUs can achieve improved performance and efficiency versus central processing units (CPUs) by dedicating a larger fraction of hardware resources to computation. In addition, their general-purpose programmability makes contemporary GPUs appealing to software developers in comparison to domain-specific accelerators. This book provides an introduction to those interested in studying the architecture of GPUs that support general-purpose computing. It collects together information currently only found among a wide range of disparate sources. The authors led development of the GPGPU-Sim simulator widely used in academic research on GPU architectures.

The first chapter of this book describes the basic hardware structure of GPUs and provides a brief overview of their history. Chapter 2 provides a summary of GPU programming models relevant to the rest of the book. Chapter 3 explores the architecture of GPU compute cores. Chapter 4 explores the architecture of the GPU memory system. After describing the architecture of existing systems, Chapters \ref{ch03} and \ref{ch04} provide an overview of related research. Chapter 5 summarizes cross-cutting research impacting both the compute core and memory system.

This book should provide a valuable resource for those wishing to understand the architecture of graphics processor units (GPUs) used for acceleration of general-purpose applications and to those who want to obtain an introduction to the rapidly growing body of research exploring how to improve the architecture of these GPUs.

Table of Contents: Preface / Acknowledgments / Introduction / Programming Model / The SIMT Core: Instruction and Register Data Flow / Memory System / Crosscutting Research on GPU Computing Architectures / Bibliography / Authors' Biographies

[PDF \(1407 KB\)](#) | [PDF Plus \(1806 KB\)](#)
[Home](#) > [Series home](#) > [Abstract](#)

[Prev. lecture](#) | [Next lecture](#)
[View/Print PDF \(1407 KB\)](#)
[View PDF Plus \(1806 KB\)](#)
[Add to favorites](#)
[Email to a friend](#)
[XML](#) | [TOC Alert](#) | [Citation Alert](#)
[What is RSS?](#)

Quick Links

- [Purchase print or personal eBook](#)
- Alert me when:
[New articles cite this article](#)
- [Download to citation manager](#)
- Related articles found in:
[Morgan & Claypool](#)
- [View Most Downloaded Articles](#)

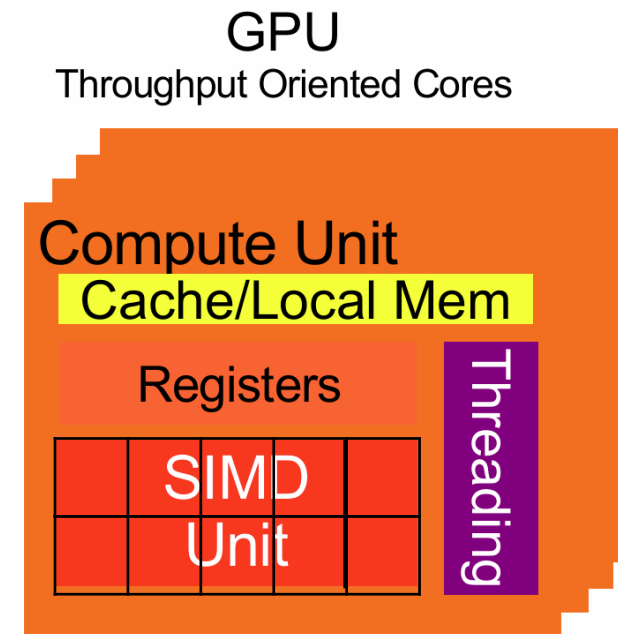
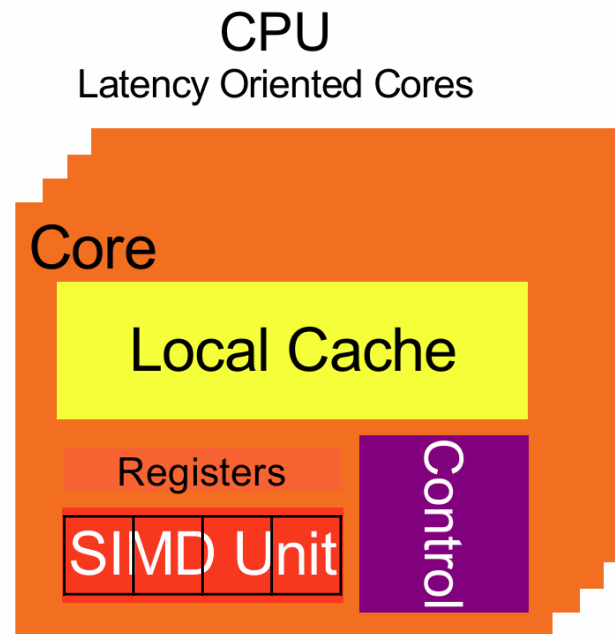
Quick Search

 for

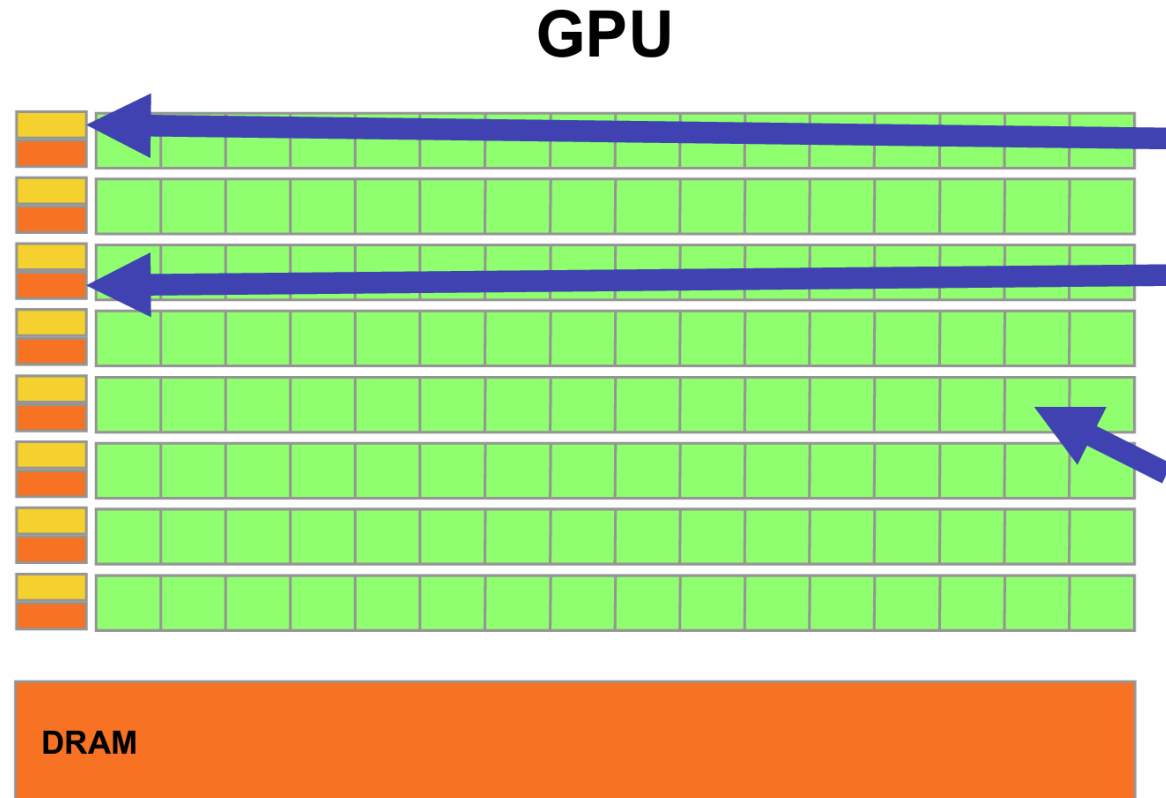
Authors:

- ☐ Tor M. Aamodt
- ☐ Wilson Wai Lun Fung
- ☐ Timothy G. Rogers

CPU and GPU are designed very differently



GPUs: Throughput Oriented Design



- › Small caches
 - › To boost memory throughput
- › Simple control
 - › No branch prediction
 - › No data forwarding
- › Energy efficient ALUs
 - › Many, long latency but heavily pipelined for high throughput
- › Require massive number of threads to tolerate latencies
 - › Threading logic
 - › Thread state

Applications Use Both CPU and GPU



- › CPUs best used for sequential parts where latency matters
- › CPUs can be >10X faster than GPUs for sequential code .
- › GPUs best used for parallel parts where throughput wins
- › GPUs can be >10X faster than CPUs for parallel code

Parallelizing code is easy. Making parallel code fast is hard.

GPU Instruction Set Architecture (ISA)

- NVIDIA defines a virtual ISA, called “PTX” (Parallel Thread eXecution)
- More recently, Heterogeneous System Architecture (HSA) Foundation (AMD, ARM, Imagination, Mediatek, Samsung, Qualcomm, TI) defined the HSAIL virtual ISA.
- PTX is Reduced Instruction Set Architecture (e.g., load/store architecture)
- Virtual: infinite set of registers (much like a compiler intermediate representation)
- PTX translated to hardware ISA by backend compiler (“ptxas”). Either at compile time (nvcc) or at runtime (GPU driver).

Some Example PTX Syntax

- Registers declared with a type:

```
.reg .pred  p, q, r;
```

```
.reg .u16   r1, r2;
```

```
.reg .f64   f1, f2;
```

- ALU operations

```
add.u32 x, y, z;           //  $x = y + z$ 
```

```
mad.lo.s32 d, a, b, c;    //  $d = a * b + c$ 
```

- Memory operations:

```
ld.global.f32 f, [a];
```

```
ld.shared.u32 g, [b];
```

```
st.local.f64 [c], h
```

- Compare and branch operations:

```
setp.eq.f32 p, y, 0;      // is y equal to zero?
```

```
@p bra L1    // branch to L1 if y equal to zero
```

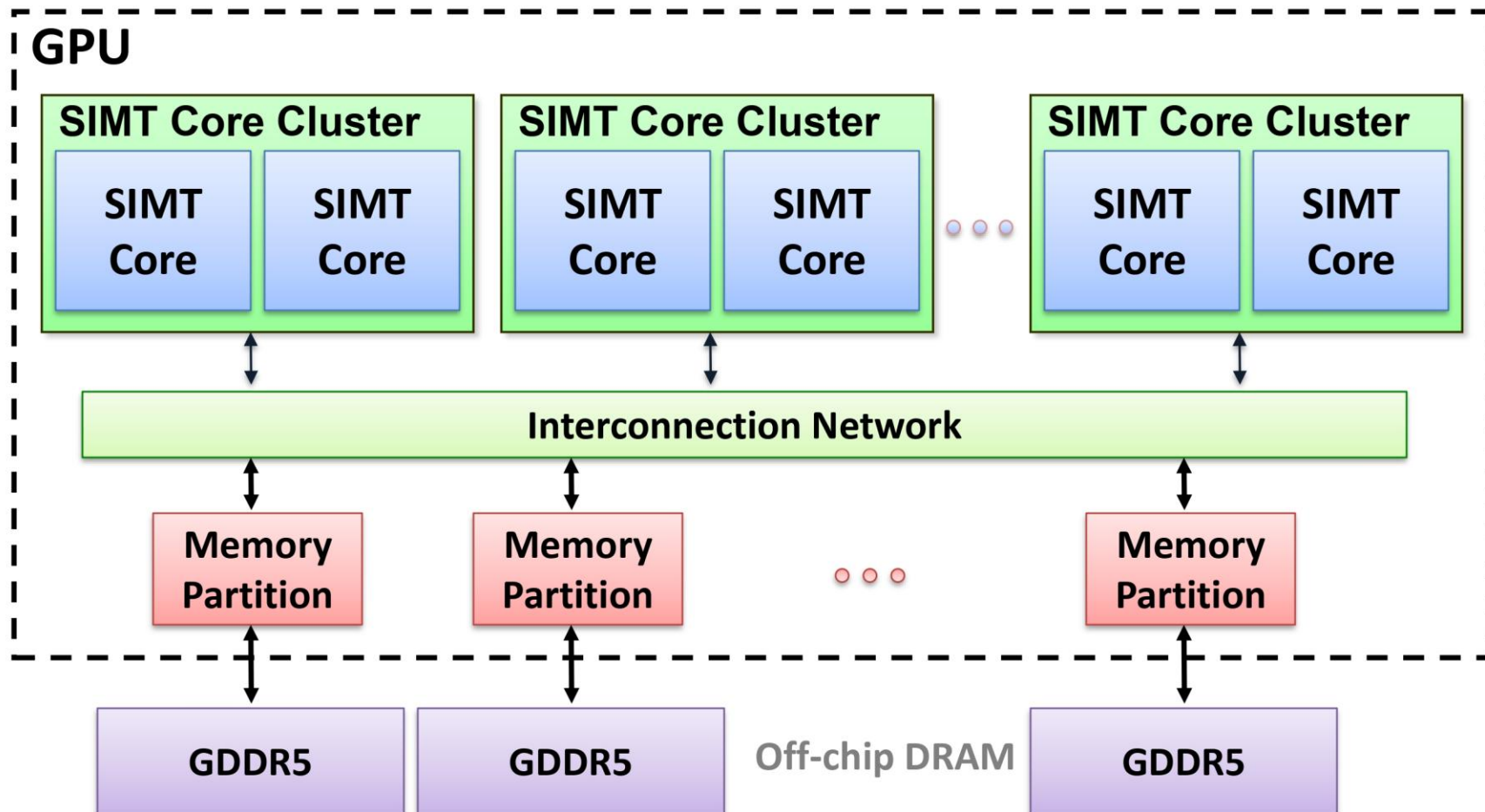
Part 2: Generic GPGPU Architecture

Extra resources

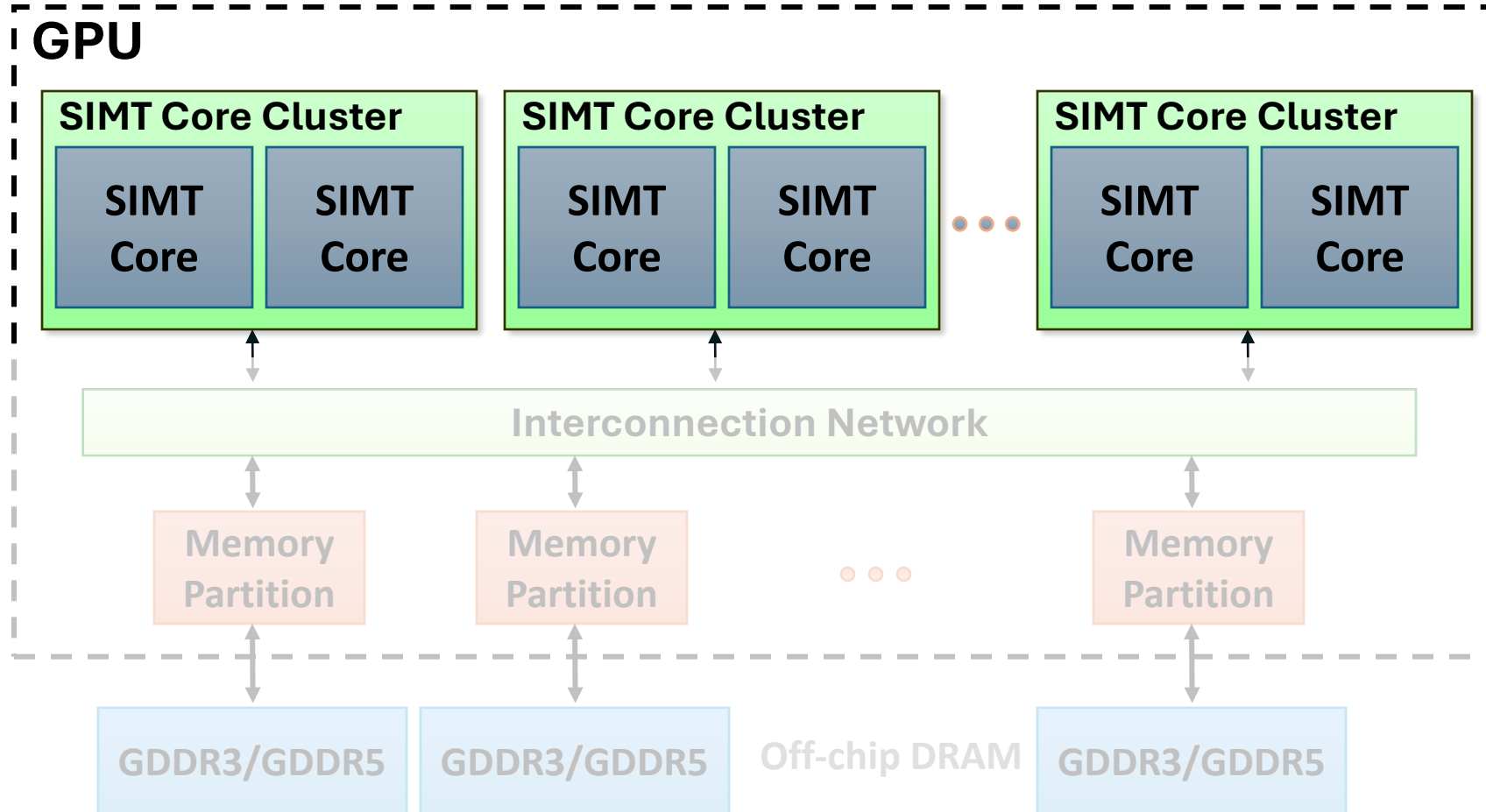
GPGPU-Sim 3.x Manual http://gpgpu-sim.org/manual/index.php/GPGPU-Sim_3.x_Manual

GPU Microarchitecture Overview

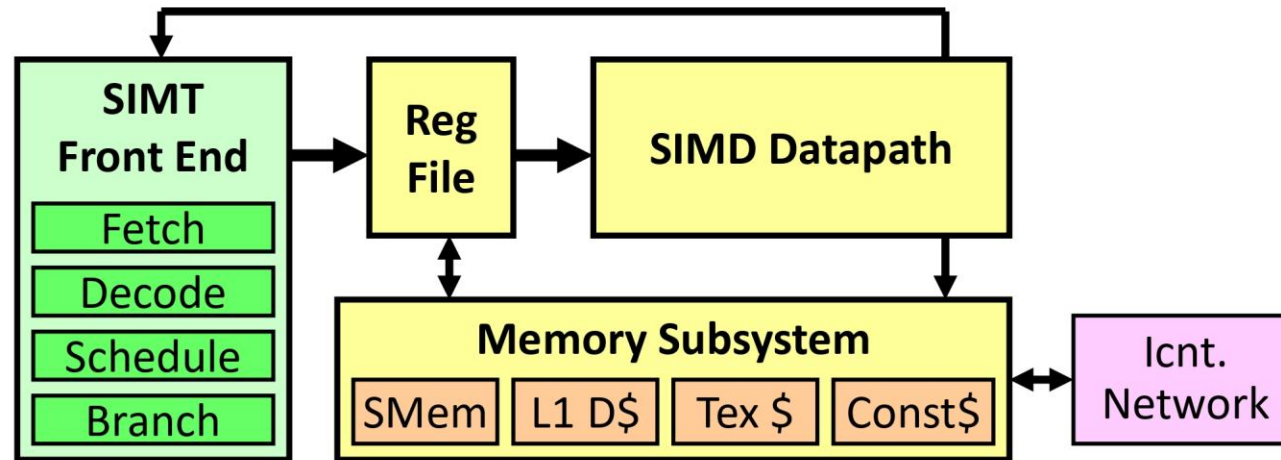
Single-Instruction, Multiple-Threads



GPU Microarchitecture Overview



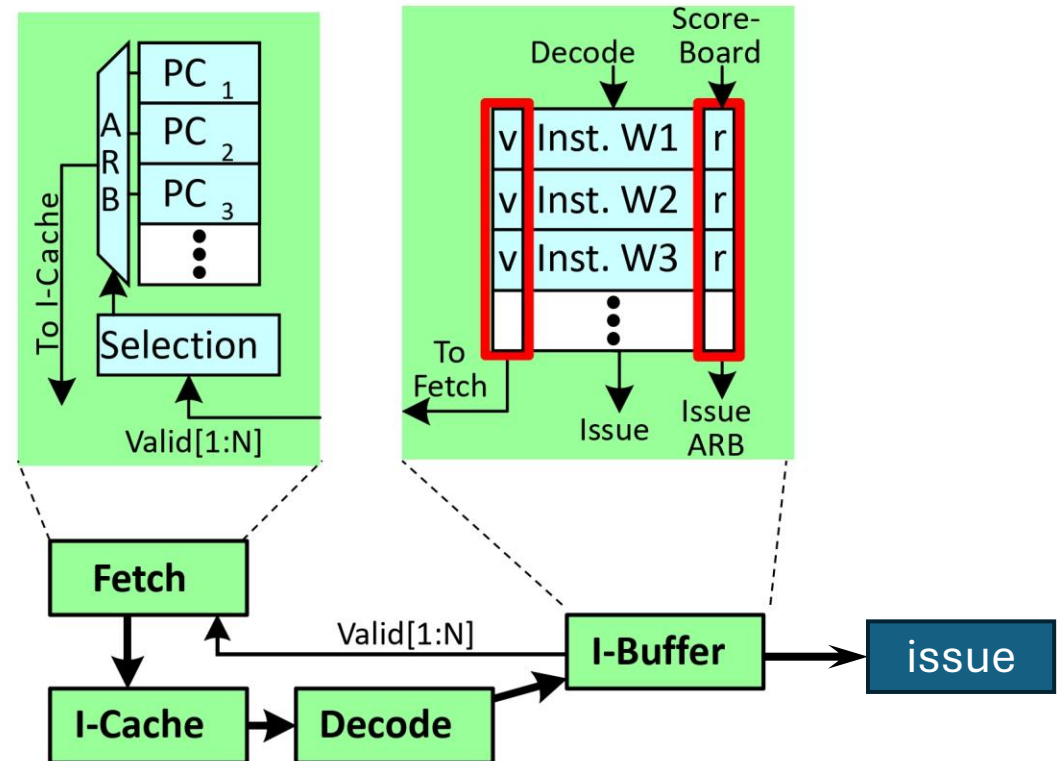
Inside a SIMT Core



- SIMT front end / SIMD backend
- Fine-grained multithreading
 - Interleave warp execution to hide latency
 - Register values of all threads stays in core

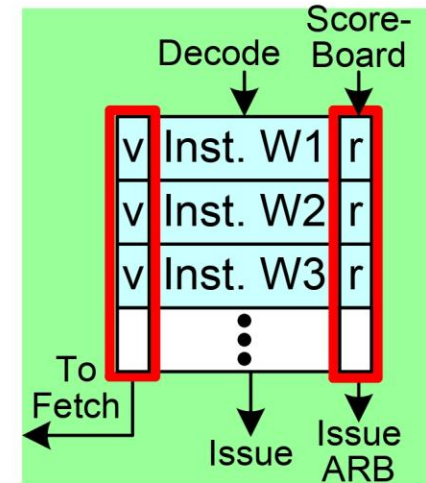
Fetch + Decode

- Arbitrate the I-cache among warps
 - Cache miss handled by fetching again later
- Fetched instruction is decoded and then stored in the I-Buffer
 - 1 or more entries / warp
 - Only warp with vacant entries are considered in fetch



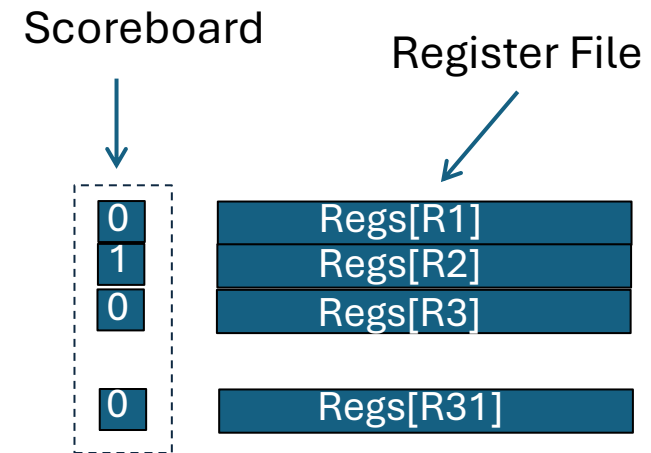
Instruction Issue

- Select a warp and issue an instruction from its I-Buffer for execution
 - Scheduling: Greedy-Then-Oldest (GTO)
 - GT200/later Fermi/Kepler: Allow dual issue (superscalar)
 - Fermi: Odd/Even scheduler
 - To avoid stalling pipeline might keep instruction in I-buffer until know it can complete (replay)



Review: In-order Scoreboard

- Scoreboard: a bit-array, 1-bit for each register
 - If the bit is *not* set: the register has valid data
 - If the bit is set: the register has stale data
i.e., some outstanding instruction is going to change it
- Issue in-order: $RD \leftarrow F_n(RS, RT)$
 - If SB[RS] or SB[RT] is set $\rightarrow$ RAW, stall
 - If SB[RD] is set $\rightarrow$ WAW, stall
 - Else, dispatch to FU (F_n) and set SB[RD]
- Complete out-of-order
 - Update GPR[RD], clear SB[RD]



Example

+

Code

```
ld  r7, [r0]
mul r6, r2, r5
add r8, r6, r7
```

Scoreboard

	Index 0	Index 1	Index 2	Index 3
Warp 0	-	-	r8	-
Warp 1	-	-	-	-

Instruction Buffer

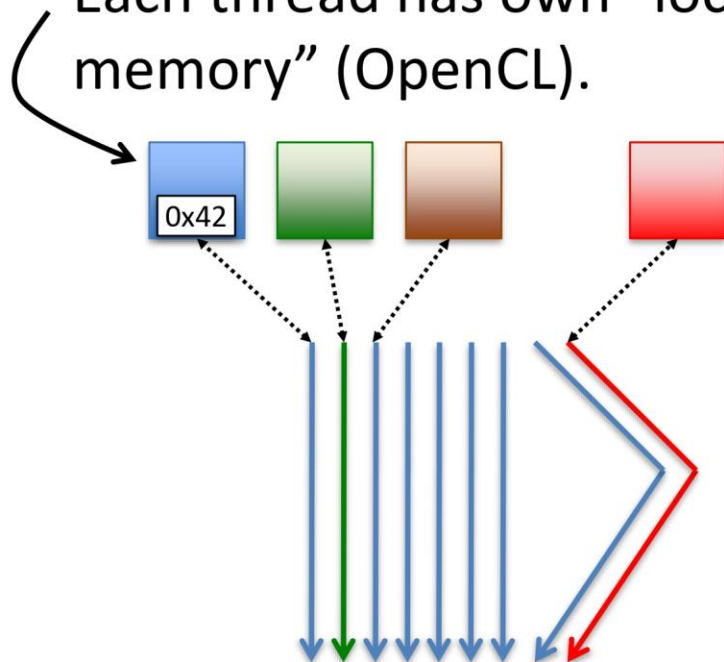
		i0	i1	i2	i3
Warp 0					
	add r8, r6, r7	0	0	0	0
Warp 1					

GPU Memory Address Spaces

- GPU has three address spaces to support increasing visibility of data between threads: local, shared, global
- In addition two more (read-only) address spaces: Constant and texture.

Local (Private) Address Space

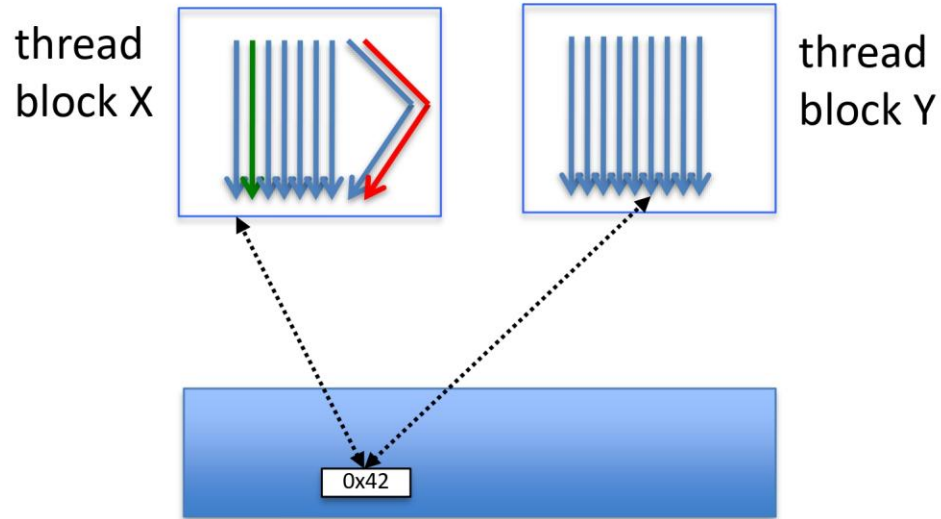
Each thread has own “local memory” (CUDA) “private memory” (OpenCL).



Note: Location at address 100 for thread 0 is different from location at address 100 for thread 1.

Contains local variables private to a thread.

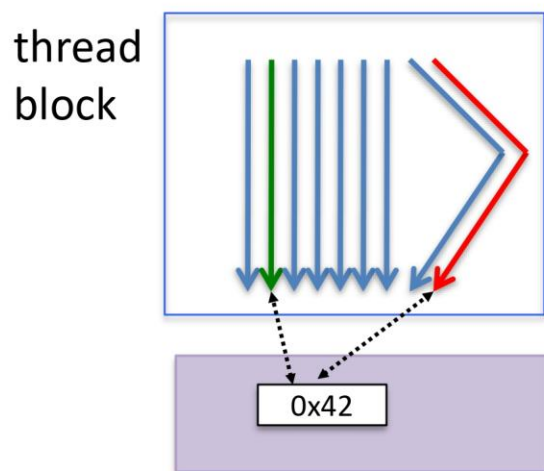
Global Address Spaces



Each thread in the different thread blocks (even from different kernels) can access a region called “global memory” (CUDA/OpenCL).

Commonly in GPGPU workloads threads write their own portion of global memory. Avoids need for synchronization—slow; also unpredictable thread block scheduling.

Shared (Local) Address Space



Each thread in the same thread block (work group) can access a memory region called “shared memory” (CUDA) “local memory” (OpenCL).

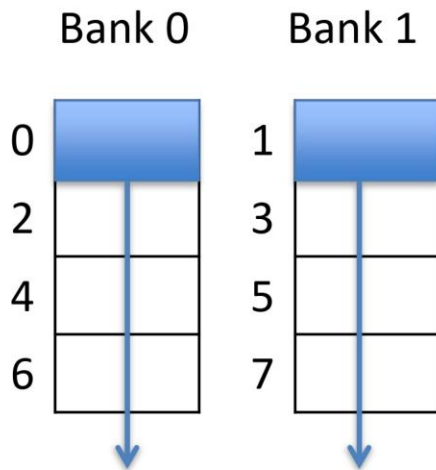
Shared memory address space is limited in size (16 to 48 KB).

Used as a software managed “cache” to avoid off-chip memory accesses.

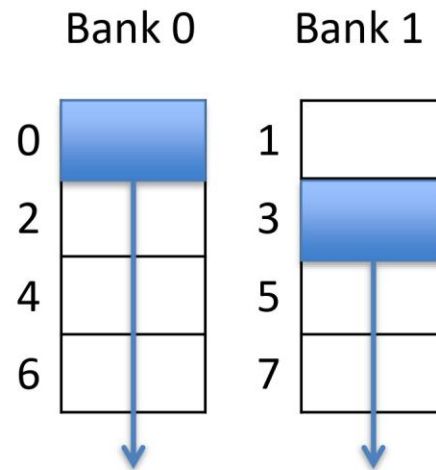
Synchronize threads in a thread block using `__syncthreads();`

Review: Bank Conflicts

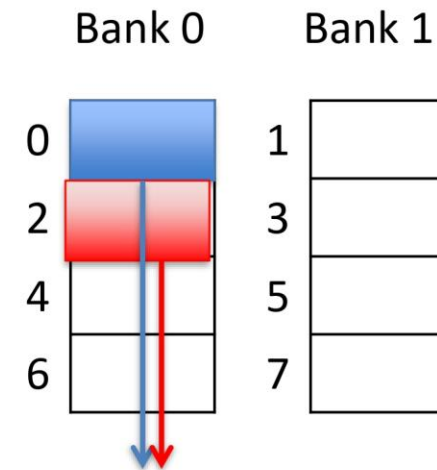
- To increase bandwidth common to organize memory into multiple banks.
- Independent accesses to different banks can proceed in parallel



Example 1: Read 0, Read 1
(can proceed in parallel)



Example 2: Read 0, Read 3
(can proceed in parallel)



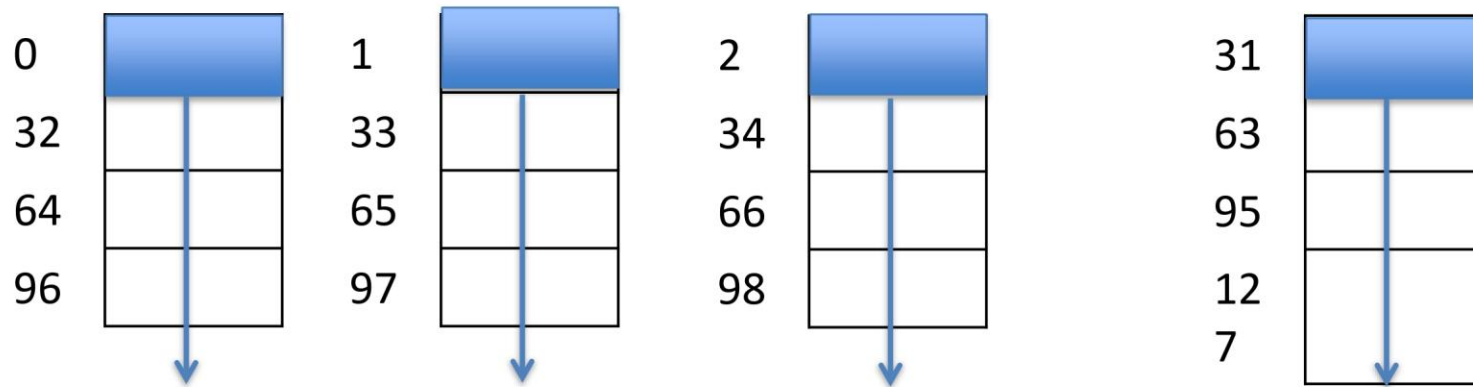
Example 3: Read 0, Read 2
(bank conflict)

Shared Memory Bank Conflicts

```
__shared__ int A[BSIZE];
```

...

```
A[threadIdx.x] = ... // no conflicts
```

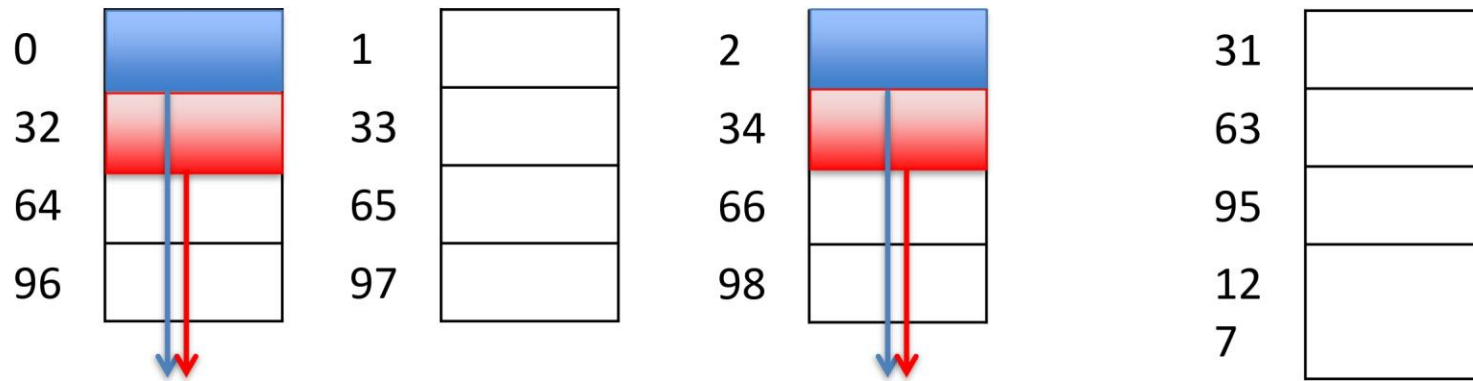


Shared Memory Bank Conflicts

```
__shared__ int A[BSIZE];
```

...

```
A[2*threadIdx.x] = // 2-way  
conflict
```

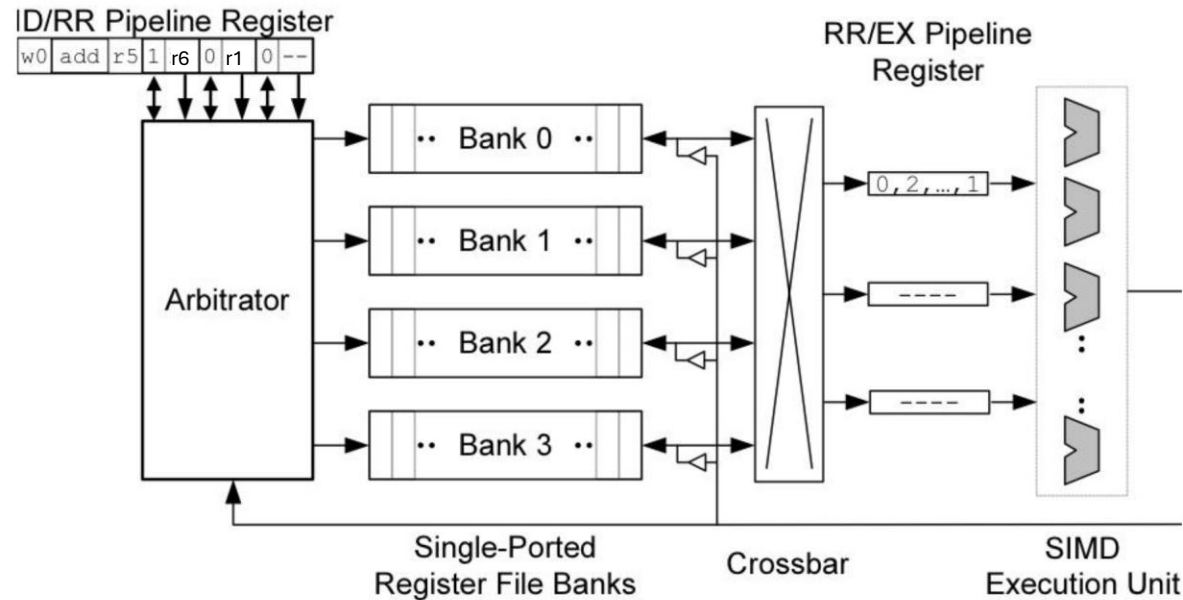


Register File

- 32 warps, 32 threads per warp, 16 x 32-bit registers per thread = **64KB register file**.
- Need “4 ports” (e.g., FMA) greatly increase area.
- Alternative: banked single ported register file. How to avoid bank conflicts?

Banked Register File

Strawman microarchitecture:



Register layout:

Bank 0	Bank 1	Bank 2	Bank 3
...	...	...	...
w1:r4	w1:r5	w1:r6	w1:r7
w1:r0	w1:r1	w1:r2	w1:r3
w0:r4	w0:r5	w0:r6	w0:r7
w0:r0	w0:r1	w0:r2	w0:r3

Register Bank Conflicts

		i1: mad r2₂, r5₁, r4₀, r6₂ i2: add r5₁, r5₁, r1₁	
Decode Cycle	Warp	Instruction	
↓	0	w3	i1: mad r2, r5, r4, r6
	1	w0	i2: add r5, r5, r1
	4	w1	i2: add r5, r5, r1

Bank 0	Bank 1	Bank 2	Bank 3
...	...	...	...
w1:r4	w1:r5	w1:r6	w1:r7
w1:r0	w1:r1	w1:r2	w1:r3
w0:r4	w0:r5	w0:r6	w0:r7
w0:r0	w0:r1	w0:r2	w0:r3

- warp 0, instruction 2 has two source operands in bank 1: takes two cycles to read.
- Also, warp 1 instruction 2 is same and is also stalled.
- Can use warp ID as part of register layout to help.

Register Bank Conflicts

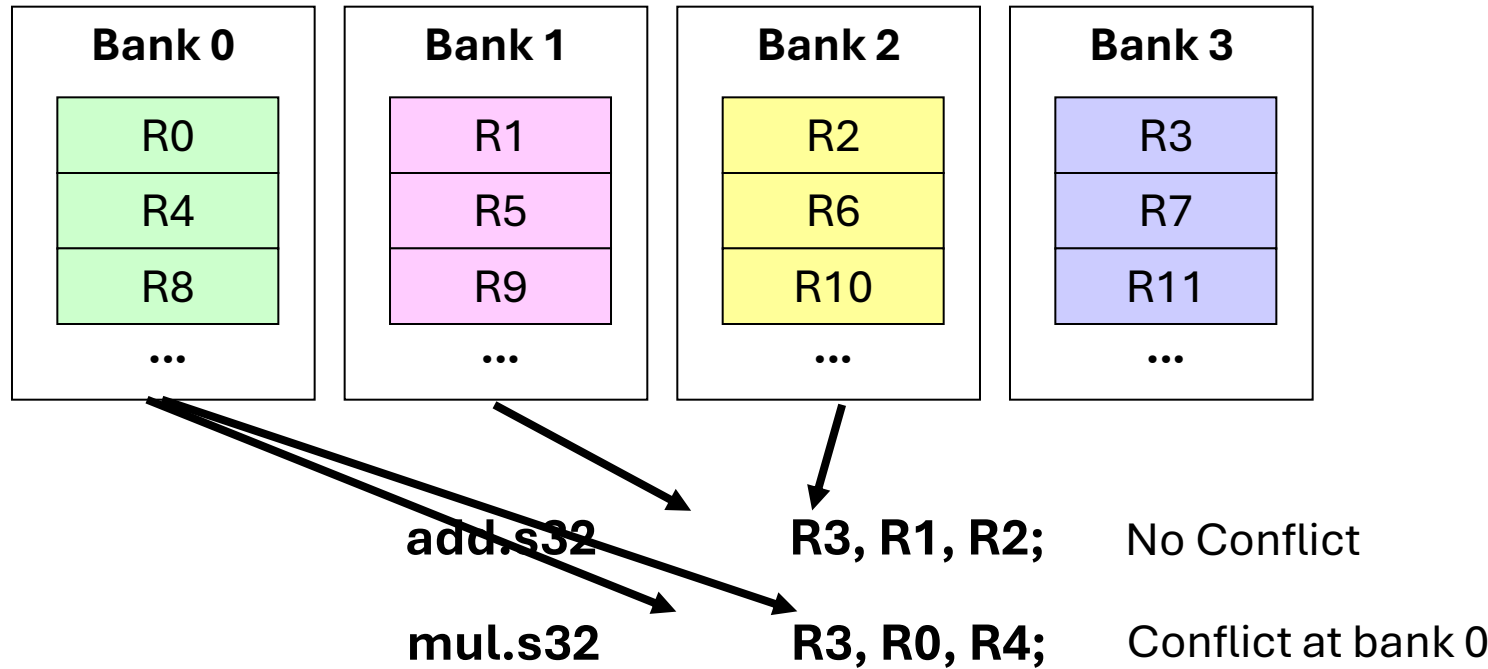
$\wedge$ = bitwise XOR

$\&$ = bitwise AND

```
bank = (reg_index ^ (warp_id & 3)) & 3 // NBANKS=4
```

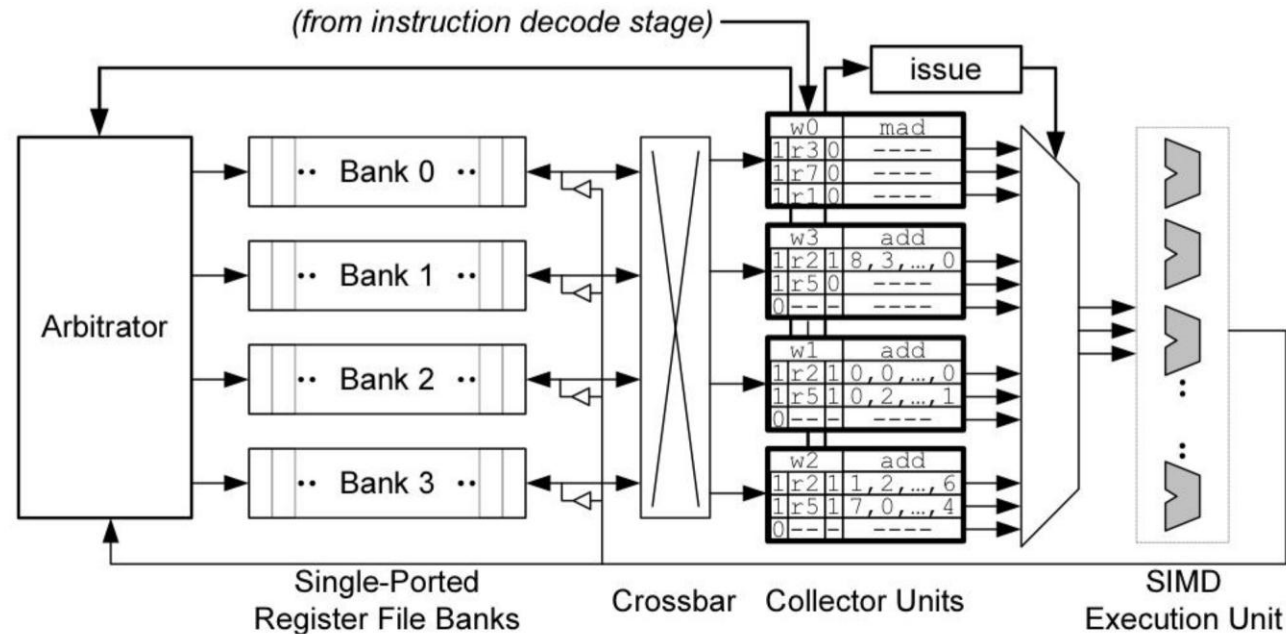
- **W0 (warp_id=0):**
 - $r5 \rightarrow (5 \wedge 0) \& 3 = 5 \& 3 = 1$
 - $r1 \rightarrow (1 \wedge 0) \& 3 = 1 \& 3 = 1 \rightarrow$ (still a **within-warp** conflict)
- **W1 (warp_id=1):**
 - $r5 \rightarrow (5 \wedge 1)=4 \rightarrow 4 \& 3 = 0$
 - $r1 \rightarrow (1 \wedge 1)=0 \rightarrow 0 \& 3 = 0 \rightarrow$ (within-warp conflict in bank 0)
- **W2 (warp_id=2):**
 - $r5 \rightarrow (5 \wedge 2)=7 \rightarrow 7 \& 3 = 3$
 - $r1 \rightarrow (1 \wedge 2)=3 \rightarrow 3 \& 3 = 3 \rightarrow$ (within-warp conflict in bank 3)
- Can use warp ID as part of register layout to help.

Operand Collector



- Term “Operand Collector” appears in figure in NVIDIA Fermi Whitepaper
- Operand Collector Architecture (US Patent: 7834881)
 - Interleave operand fetch from different threads to achieve full utilization

Operand Collector

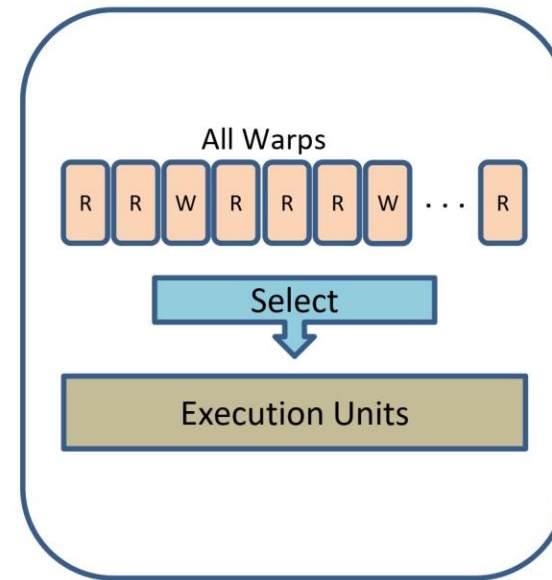


- Issue instruction to collector unit.
- Collector unit similar to reservation station in tomasulo's algorithm.
- Stores source register identifiers.
- Arbiter selects operand accesses that do not conflict on a given cycle.
- Arbiter needs to also consider writeback (or need read+write port)

Warp Scheduling Basics

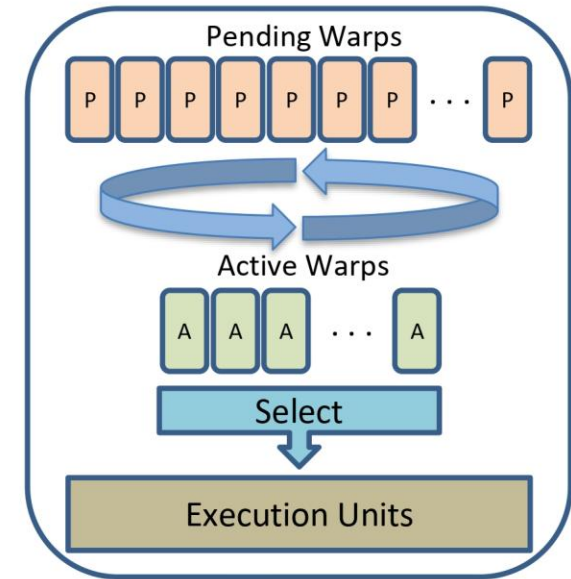
Loose Round Robin (LRR)

- Goes around to every warp and issue if ready (R)
- If warp is not ready (W), skip and issue next ready warp
- Issue: Warps all run at the same speed, potentially all reaching memory access phase together and stalling.



Two-level (TL)

- Warps are grouped into two groups:
 - Pending warps
(potentially waiting on long latency instr.)
 - Active warps
(Ready to execute)
 - Warps move between Pending and Active groups
- Within the Active group, issue LRR
- Goal: Overlap warps performing computation with warps performing memory access



Greedy-then-oldest (GTO)

- Schedule from a single warp until it stalls
- Then pick the oldest warp (time warp assigned to core)
- Goal: Improve cache locality for greedy warp

